

北大家园食堂模拟器作业报告

一、项目概述

本项目的题目是“北大家园食堂模拟器”。受到天天去家园食堂吃午饭的启发，我们把同学们比较熟悉的校园食堂体验做成一个有流程、有节奏的桌面端经营游戏。游戏场景、食材都取材于北京大学家园食堂：玩家从采购食材开始，进入营业区后需要在不同窗口之间切换，完成备餐、接单、出餐和日结。整体体验接近“食堂窗口营业的一天”，既有经营类游戏中的资金、库存和订单，也有拖拽食材、等待熟成、避免过火等即时操作。

项目使用 Python 和 PySide6 实现。界面主体采用 Qt Graphics View 体系：静态图片资源由场景配置文件统一加载，动态按钮、进度条、订单小票和状态提示由程序运行时绘制。这样做的好处是界面看起来更接近游戏画面，同时又可以保留 Qt 在事件、窗口、信号槽和组件管理上的优势。

项目最终代码放在仓库的 `pku_simulator/` 文件夹中，主要运行入口是：

```
cd pku_simulator
python -m pip install -e .
python run_qt.py
```

二、程序功能介绍

1. 开始页与采购流程

游戏开始后，玩家首先进入开始页。开始页会展示当前营业日、账户余额和出餐仓数量。点击“开始营业”后进入采购商店。商店中提供肉类、蔬菜、主食、烧烤串、调料等多种食材。玩家可以根据余额购买 1 份或 5 份食材，购买后库存增加、余额减少。

采购流程中有两个细节：

第一，购买不是只改界面数字，而是经过业务服务层统一处理。按钮本身只提供 `buy:食材:数量` 这样的点击标记，真正的扣钱、入库和存档由 `GameService.buy_ingredient()` 完成。

第二，营业中也可以从底部导航返回商店补货，但营业中补货会使用更高的价格倍率。这样设计是为了让玩家在开局采购时需要做一点计划，而不是完全依赖营业中临时补货。

2. 营业区与工位切换

进入营业区后，玩家可以在五个主要场景之间切换：

- 烤盘间
- 肠粉台
- 主食区
- 烧烤架
- 点餐台

底部 dock 导航负责切换这些场景。代码中，每个场景都有一个 `scene_key`，例如 `griddle_station`、`cheung_fun_station`、`bbq_station`。点击底部图标时，点击路由会解析 `dock:scene_key`，再调用 `WorkbenchPage.switch_to_scene()` 切换后台状态，并通知 `GameSceneView` 重新加载对应场景。

3. 烤盘间

烤盘间主要使用拖拽操作。玩家把食材拖到烤盘上后，程序会记录食材种类和放入时间。烤盘上的食材需要等待一段时间才算熟，如果超过过火时间，烤盘会损坏，当天不能继续使用。点击烤盘可以打开信息面板，面板中显示每个食材的熟度进度，并提供“出锅”按钮。

烤盘出锅时会根据烹饪时间判断是否可食用，再把菜品写入出餐仓。出餐仓中的菜品之后会被点餐台订单匹配使用。

4. 肠粉台

肠粉台的操作比烤盘更复杂一些。玩家需要先用面糊勺取面糊，再倒到指定烤台上，然后可以加入鸡蛋、鲜虾等配料。配料同样有熟成时间，时间过长会导致烤台损坏。肠粉做好后，需要使用铲勺完成处理，再从信息面板中出锅。

这个工位的设计目的是让不同工位不只是换一套图片，而是有不同的操作顺序。肠粉台强调“先铺底、再加料、再处理”的流程；烤盘间则强调“多食材同时计时”。

5. 主食区

主食区围绕电饭煲设计。玩家需要向电饭煲中加入米和水，两者都加入后才能开始蒸饭。蒸饭需要等待计时完成，完成后可以出锅，米饭进入出餐仓。

这个工位相对稳定，主要承担订单中主食类需求，也让玩家在紧张的烹饪工位之外处理一个偏流程管理的任务。

6. 烧烤架

烧烤架支持拖拽生烤串到槽位。每个槽位只能放一串，放上后开始计时。烤熟后点击烤串会出现出串面板。烧烤还支持孜然和辣椒调味，其中孜然作为默认调味，辣椒可以通过拖拽调料添加。

烧烤架的界面中加入了烟雾效果和熟串提示，主要是为了让画面更有“正在营业”的动态感。

7. 点餐台与订单系统

点餐台是整个经营流程的闭环。营业过程中，服务层会按时间生成顾客订单。顾客出现在点餐台后，玩家需要点击顾客、接单、阅读小票，之后订单会挂在顶部等待出餐。玩家点击挂起的小票后，可以查看订单详情。如果出餐仓中的菜品能够满足订单要求，“出餐”按钮会启用；否则按钮保持不可用。

订单匹配由 `GameService.preview_order_match()` 完成。真正出餐时调用

`GameService.serve_order()`：程序会计算收入、处理不完全匹配或不可食用食物、移除已经使用的菜品，并把收入变化显示在点餐台。

8. 日结、存档与失败条件

一天营业结束后，系统会进行日结。日结时会清理过期食材、处理设备维修费用，并进入下一天。如果余额低于破产线，则进入失败页。游戏状态会保存到 JSON 文件中，开发运

运行时存档位于项目内的 `save/game_state.json`，打包运行时会写到用户目录，避免应用内部不可写。

三、项目模块与类设计细节

项目在后期整理时把原来比较集中的代码拆成了多个模块，主要目标是让界面、事件、业务状态和资源配置之间的边界更清楚。

1. 启动与主窗口

项目入口是 `run_qt.py` 和 `pku_simulator.main`。入口只负责创建 `QtApp` 并进入 Qt 主循环。真正的主窗口装配在 `qt_app.py` 中完成。

核心类：

- `QtApp`：创建 `QApplication`，持有 `MainWindow`。
- `MainWindow`：主窗口类，负责加载存档、创建 `GameService`、创建 `GameSceneView`、创建页面栈，并连接图形场景发出的各种信号。

`MainWindow` 使用多个 `mixin` 组合功能，例如导航、点击路由、商店控制、点餐台控制、BGM、拖拽投放处理等。这样做可以避免所有功能都堆在一个大类文件里。

2. 图形场景层

图形场景由 `GameSceneView` 管理。它继承了 `QGraphicsView`，并组合了两个主要 `mixin`：

- `SceneDrawingMixin`：提供添加图片、文字、矩形、圆角矩形、椭圆等绘制方法。
- `SceneDragMixin`：处理鼠标按下、移动、释放，以及拖拽命中判断。

`GameSceneView` 的职责是加载和展示场景，而不是直接修改游戏业务数据。它在合适的时候发出信号：

```
asset_clicked(str)
griddle_ingredient_dropped(str, int)
cheung_fun_ingredient_dropped(str, int)
```

```
bbq_skewer_dropped(str, int)
staple_ingredient_dropped(str, int)
```

也就是说，图形层只回答“玩家点了什么、拖到了哪里”，后续怎么扣库存、怎么改变工位状态，由控制器和业务层处理。

3. 点击路由

点击事件集中在 `qt_click_router.py` 的 `GraphicalClickRouterMixin` 中处理。项目中所有可点击图形项都会设置一个稳定的 `data` 字符串，例如：

```
buy:egg:1
dock:bbq_station
griddle_out:0
bbq_finish:2
counter_serve:5
```

点击路由根据这些前缀判断下一步调用哪个控制器方法。这种设计让 `overlay` 绘制代码和业务逻辑解耦：绘制层只需要画按钮并写上 `data`，不用知道购买、出餐、切场景的具体实现。

4. 拖拽投放控制

拖拽命中首先由 `qt_scene_drag.py` 判断。例如，玩家拖动烤盘食材，释放鼠标时程序会判断释放点是否落在某个烤盘区域。如果命中，则发出 `griddle_ingredient_dropped` 信号；如果没有命中，则播放回弹或丢进垃圾桶的动画。

信号会进入 `qt_workbench_drops.py` 的 `WorkbenchDropControllerMixin`。这里才开始处理业务：

- 检查当前是否在营业区。
- 检查食材是否合法。
- 调用 `GameService.consume_ingredient()` 扣库存。
- 调用对应工作台方法修改状态。
- 刷新状态栏和图形场景。

这样拆分后，拖拽动画、投放命中和业务扣库存不会互相混在一起。

5. 营业区总控与工作台 mixin

营业区总控类是 `WorkbenchPage`。它继承了四个工位 mixin：

- `GriddleStationMixin`
- `CheungFunStationMixin`
- `StapleStationMixin`
- `BbqStationMixin`

`WorkbenchPage` 负责协调营业区生命周期，例如进入营业区时启动订单计时器，离开时停止计时器；每秒推进订单；高频刷新工位信息面板；在营业日结束时触发日结。

各个工作台 mixin 只维护本工位状态。例如：

- 烤盘维护每个烤盘中的食材、放入时间、是否损坏。
- 肠粉维护面糊勺、烤台、配料和爆炸状态。
- 主食维护电饭煲中是否有米、水、是否开始蒸饭。
- 烧烤维护槽位中的烤串、开始时间、调料状态。

这种设计让每个工位可以独立调整，不会因为修改烧烤逻辑影响肠粉台。

6. Overlay 绘制层

项目的动态图层集中在 `qt_overlays_*.py`：

- `qt_overlays_start_shop.py`：开始页、商店、底部 dock、音乐设置面板。
- `qt_overlays_workbench.py`：四个工作台的信息面板、进度条、动态素材。
- `qt_overlays_counter.py`：顾客、小票、出餐按钮、现金提示。
- `qt_overlays.py`：根据当前场景分发到具体 overlay。

Overlay 层原则上只读状态并绘制界面，不直接做业务修改。比如点餐台放大小票时，overlay 会调用 `preview_order_match()` 判断能否出餐，然后决定按钮是否可点击；真正点击出餐后，仍然由点击路由转到服务层。

7. 服务层与核心数据

业务服务集中在 `services/game_service.py`。它负责：

- 购买食材。
- 消耗库存。
- 记录成品菜。
- 生成和推进订单。
- 预览订单匹配。
- 完成出餐。
- 日结、维修扣费和破产判断。

核心数据在 `core/` 中：

- `models.py`：定义 `GameState`、`IngredientDef`、`InventoryBatch`。
- `inventory.py`：库存批次管理，支持先进先出消耗和过期清理。
- `catalog.py`：食材目录。
- `save_repo.py`：存档读写。
- `config.py`：路径、窗口尺寸、画布尺寸等配置。

在设计上，Qt 页面尽量不直接修改 `GameState` 字段，而是通过 `GameService` 或工位 `mixin` 间接修改。这样更容易保证扣钱、扣库存、保存状态这些操作不会漏掉。

8. 资源布局管理

图片资源的位置和层级没有硬编码在 Python 文件中，而是统一写在 `assets/asset_positions.json`。运行时由 `AssetLayoutStore` 读取这个文件，`GameSceneView` 再按当前场景加载资源。

项目中还保留了一个布局编辑器，可以在开发模式下打开。它支持拖拽素材、调整坐标、尺寸、层级、旋转和透视参数，然后写回 JSON。这样李铭月在整理图片资产时，可以比较直观地调整素材在画面中的位置，不需要每次都改代码。

四、界面与创意设计

项目创意主要来自北京大学家园食堂。我们希望玩家一看到场景和食材，就能联想到校园食堂窗口：采购、备餐、排队点餐、窗口出餐，这些都是同学们日常熟悉的片段。相比完全虚构的餐厅经营游戏，这个主题更有校园感，也更适合作为 Qt 大作业展示。

界面上，我们没有完全使用普通按钮和表单，而是以大画布场景为主。不同工位有不同背景和素材，玩家通过拖动图片中的食材完成操作。按钮、提示框、订单小票、进度条等由程序绘制在场景上。这样既能体现 Qt Graphics View 的使用，也能让游戏界面比普通窗口应用更有表现力。

图片资产主要通过 gpt-image-2 生成，再经过人工筛选、命名和定位。生成图片并不是直接放进项目就结束，还需要考虑素材大小、透明背景、视角一致性、图层遮挡、不同状态的对应关系等问题。比如烧烤串需要区分生熟状态和左右位置，肠粉需要区分不同配料组合，点餐台需要顾客和小票状态。最终这些素材通过 `asset_positions.json` 组织起来，方便程序按场景加载。

五、小组成员分工

赫震霖

主要负责游戏逻辑代码和整体程序结构。具体包括：

- 搭建 PySide6 项目入口、主窗口和页面切换逻辑。
- 实现 `GameSceneView` 的场景加载、点击信号和拖拽交互。
- 设计并实现商店购买、库存、订单、出餐、日结、存档等业务逻辑。
- 实现烤盘、肠粉、主食、烧烤和点餐台各工位的状态管理。
- 整理代码结构，把原来较集中的功能拆分为主窗口、场景、控制器、工作台、服务层和核心数据层。
- 编写 README，整理 GitHub 提交目录，处理运行和打包脚本。

李铭月

主要负责图片资产生成和场景资源整理。具体包括：

- 使用 gpt-image-2 生成游戏所需图片资产，包括食材、厨具、顾客、场景背景和工位状态图。
- 根据游戏主题筛选更符合“北大家园食堂”风格的素材，保持整体视觉风格尽量统一。
- 整理素材命名和分类，区分食材图片、厨具图片、功能性图片和场景背景。
- 使用布局编辑器调整图片资产的位置、尺寸和图层，将结果写入 `asset_positions.json`。
- 配合代码逻辑补充不同状态下需要的素材，例如熟烤串、损坏烤盘、电饭煲状态、不同配料肠粉等。

整体上，赫震霖更偏程序逻辑和架构实现，李铭月更偏视觉资产和资源定位。两部分需要反复配合，因为很多玩法不是单靠代码或图片一边就能完成：代码需要知道素材名称和状态，素材也需要按代码中的状态组合准备。

六、项目总结与反思

这次项目最大的收获是，我们从一个看起来比较简单的“食堂经营游戏”里实际做出了较完整的状态流。采购、库存、烹饪、出餐、订单、日结之间是连在一起的，玩家前面买什么、做什么，会影响后面能不能完成订单。这个闭环让项目比单纯的界面展示更像一个真正可以玩的小游戏。

实现过程中比较困难的地方有三个。

第一是 Qt 图形界面和游戏逻辑之间的边界。一开始如果把点击、拖拽、业务判断都写在一个文件里，功能能跑，但后期很难维护。后来我们把 `GameSceneView`、点击路由、拖拽投放控制、工作台状态和 `GameService` 分开，代码才变得比较清楚。

第二是资源管理。图片素材数量变多后，坐标和层级很容易混乱。如果每个位置都写死在代码里，调整一次画面会非常麻烦。因此我们把素材位置放到 JSON 中，并做了布局编辑器。这个工具虽然只是开发辅助，但对后期调整界面帮助很大。

第三是不同工位的状态设计。四个工位如果逻辑完全一样，游戏会显得单调；但如果每个工位都写得太复杂，代码又会失控。最后我们给每个工位保留一个主要特点：烤盘强调熟

度和过火，肠粉强调操作顺序，主食强调组合条件和等待，烧烤强调槽位和调味。这样既有差异，也还能控制实现规模。

对于工具使用，我们在图片资产生成上使用了 gpt-image-2，在代码整理和文档润色过程中也参考了辅助工具。但最终项目的玩法设计、模块拆分、调试和整合都需要人工判断。特别是 Qt 信号连接、拖拽命中、订单匹配和素材定位这些部分，不是简单生成代码就能直接完成，需要不断运行、发现问题、调整状态和界面。

如果后续继续完善，我们希望增加更多校园化的细节，例如更丰富的顾客、更多家园食堂特色菜、不同时间段的客流变化，以及更完整的新手提示。同时，也可以给 GameService 中的订单匹配、库存和日结逻辑补充自动化测试，让项目在继续扩展时更稳。

总体来说，这次作业让我们对 Qt 的事件机制、图形场景、信号槽和项目结构有了更具体的理解。相比只做一个普通表单程序，这个项目更接近一个小型桌面游戏，也让我们体会到界面、资源和业务逻辑之间需要反复协调。